
```
def perturb(cities, itinerary):
    neighborids1 = math.floor(np.random.rand() * (len(itinerary)))
    neighborids2 = math.floor(np.random.rand() * (len(itinerary)))

    itinerary2 = itinerary.copy()

    itinerary2[neighborids1] = itinerary[neighborids2]
    itinerary2[neighborids2] = itinerary[neighborids1]

    distance1 = howfar(genlines(cities, itinerary))
    distance2 = howfar(genlines(cities, itinerary2))

    itinerarytoreturn = itinerary.copy()

    if(distance1 > distance2):
        itinerarytoreturn = itinerary2.copy()

    return(itinerarytoreturn.copy())
```

***Listing 6-3:** A function that makes a small change to an itinerary, compares it to the original itinerary, and returns whichever itinerary is shorter*

The `perturb()` function takes any list of cities and any itinerary as its arguments. Then, it defines two variables: `neighborids1` and `neighborids2`, which are randomly selected integers between 0 and the length of the itinerary. Next, it creates a new itinerary called `itinerary2`, which is the same as the original itinerary except that the cities at `neighborids1` and `neighborids2` have switched places. Then it calculates `distance1`, the total distance of the original itinerary, and `distance2`, the total distance of `itinerary2`. If `distance2` is smaller than `distance1`, it returns the new itinerary (with the switch). Otherwise, it returns the original itinerary. So we send an itinerary to this function, and it always returns an itinerary either as good as or better than the one we sent it. We call this function `perturb()` because it perturbs the given itinerary in an attempt to improve it.

Now that we have a `perturb()` function, let's call it repeatedly on a random itinerary. In fact, let's call it not just one time but 2 million times in an attempt to get the lowest traveling distance possible: